

Lab 8: Introduction to OpenGL

OpenGL (Open Graphics Library) is a cross-platform API used for rendering 2D and 3D graphics. It provides a set of functions for interacting with the GPU to create visually rich applications.

Callback Functions

Callback functions in OpenGL handle user input and system events, such as keyboard presses, mouse movements, and window resizing. Common callbacks include `glutDisplayFunc()`, `glutReshapeFunc()`, and `glutKeyboardFunc()`.

Color Commands

OpenGL provides functions to define colors using RGB(A) values. The primary function for setting colors is `glColor3f(r, g, b)` for RGB and `glColor4f(r, g, b, a)` for RGBA (with transparency).

Drawing Pixels, Lines, and Polygons

- **Pixels:** Individual pixels can be drawn using `glDrawPixels()`.
- **Lines:** OpenGL allows drawing lines with `glBegin(GL_LINES)` and `glVertex()` commands.
- **Polygons:** Shapes like triangles and quadrilaterals are drawn using `glBegin(GL_TRIANGLES)` or `glBegin(GL_POLYGON)`, followed by `glVertex()` calls.

Viewing and Lighting

- **Viewing:** The OpenGL viewing pipeline involves setting up a camera using functions like `gluLookAt()` and defining a projection using `glOrtho()` or `glFrustum()`.
- **Lighting:** OpenGL supports real-time lighting effects through `glEnable(GL_LIGHTING)` and various light sources configured with `glLightfv()` to simulate realistic shading and reflections.

OpenGL in Python

Working with OpenGL

To start download : <https://www.lfd.uci.edu/~gohlke/pythonlibs/#pyopengl>

- ✓ PyOpenGL-3.1.6-cp311-cp311-win_amd64.whl
- ✓ PyOpenGL_accelerate-3.1.6-cp311-cp311-win_amd64.whl

The install it using pip

Question 1: Callback in OpenGL

```
from OpenGL.GL import *
from OpenGL.GLUT import *
import sys

window_width = 800
window_height = 600

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glBegin(GL_TRIANGLES)
    glVertex2f(-0.5, -0.5)
    glVertex2f(0.5, -0.5)
    glVertex2f(0.0, 0.5)
    glEnd()
    glFlush()

def reshape(width, height):
    global window_width, window_height
    window_width = width
    window_height = height
    glViewport(0, 0, width, height)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glOrtho(-1, 1, -1, 1, -1, 1)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

def keyboard(key, x, y):
    if key == b'\x1b': # Escape key
        glutLeaveMainLoop()

def mouse(button, state, x, y):
    if button == GLUT_LEFT_BUTTON and state == GLUT_DOWN:
        print("Left mouse button clicked at ({}, {})".format(x, y))

def idle():
    glutPostRedisplay()

def main():
```

```

glutInit(sys.argv)
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
glutInitWindowSize(window_width, window_height)
glutCreateWindow(b"OpenGL Callbacks")

glClearColor(0.0, 0.0, 0.0, 1.0)

# Registering callbacks
glutDisplayFunc(display)
glutReshapeFunc(reshape)
glutKeyboardFunc(keyboard)
glutMouseFunc(mouse)
glutIdleFunc(idle)

glutMainLoop()

if __name__ == "__main__":
    main()

```

Question 2: Drawing Pixel in OpenGL

```

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.raw.GLU import gluOrtho2D

def draw_pixel(x, y):
    glBegin(GL_POINTS)
    glVertex2f(x, y)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glColor3f(1.0, 1.0, 1.0) # Set color to white

    draw_pixel(100, 100)
    glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(400, 400)
    glutCreateWindow(b"Drawing Pixels")

    glClearColor(0.0, 0.0, 0.0, 1.0)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, 400, 0, 400)

    glutDisplayFunc(display)
    glutMainLoop()

if __name__ == "__main__":
    main()

```

Question 3: Drawing a line in OpenGL

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.raw.GLU import gluOrtho2D

def draw_line(x1, y1, x2, y2):
    glBegin(GL_LINES)
    glVertex2f(x1, y1)
    glVertex2f(x2, y2)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glColor3f(1.0, 1.0, 1.0) # Set color to white
    draw_line(100, 100, 300, 300)

    glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(400, 400)
    glutCreateWindow(b"Drawing Lines")

    glClearColor(0.0, 0.0, 0.0, 1.0)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, 400, 0, 400)

    glutDisplayFunc(display)
    glutMainLoop()

if __name__ == "__main__":
    main()
```

Question 4: Drawing polygon in OpenGL

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.raw.GLU import gluOrtho2D

def draw_polygon():
    glBegin(GL_POLYGON)
    glVertex2f(100, 100)
    glVertex2f(300, 100)
    glVertex2f(200, 300)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    glColor3f(1.0, 1.0, 1.0) # Set color to white
```

```

draw_polygon()

glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(400, 400)
    glutCreateWindow(b"Drawing Polygons")

    glClearColor(0.0, 0.0, 0.0, 1.0)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, 400, 0, 400)

    glutDisplayFunc(display)
    glutMainLoop()

if __name__ == "__main__":
    main()

```

Question 5: Viewing & Lighting in OpenGL

```

from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.raw.GLU import gluPerspective, gluLookAt

def init():
    glEnable(GL_DEPTH_TEST)
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glLightfv(GL_LIGHT0, GL_POSITION, [0.0, 1.0, 1.0, 0.0])
    glEnable(GL_COLOR_MATERIAL)

def display():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
    gluLookAt(0.0, 0.0, 5.0,
              0.0, 0.0, 0.0,
              0.0, 1.0, 0.0)

    glutSolidSphere(1.0, 50, 50)
    glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH)
    glutInitWindowSize(400, 400)
    glutCreateWindow(b"Viewing and Lighting in OpenGL")
    glClearColor(0.0, 0.0, 0.0, 1.0)
    glMatrixMode(GL_PROJECTION)
    gluPerspective(45, 1.0, 0.1, 100)
    init()

```

```
glutDisplayFunc(display)
glutMainLoop()

if __name__ == "__main__":
    main()
```

Conclusion

- What did u learn?